Exercise 2 – Data Preprocessing

Using the given CEREALS dataset, perform data preprocessing and answer the following

questions.

1) Create a table with the 5-number summary of all the numeric attributes.

In [1]:
```
import pandas as pd


df = pd.read_csv(r"C:\Prashma\AI-ML Data set\Cereals1.csv")

summary = df.describe().loc[['min', '25%', '50%', '75%', 'max']]

summary.index = ['Min', 'Q1 (25%)', 'Median (50%)', 'Q3 (75%)', 'Max']

print("5-Number Summary Table:")
print(summary)
```

```
5-Number Summary Table:
              calories  protein  fat  sodium  fiber  carbo  sugars  potass  \
Min               50.0      1.0  0.0     0.0   0.00   -1.0    -1.0    -1.0
Q1 (25%)         100.0      2.0  0.0   132.5   0.75   12.0     3.0    40.0
Median (50%)     110.0      2.5  1.0   180.0   1.75   14.5     7.0    90.0
Q3 (75%)         110.0      3.0  2.0   212.5   3.00   17.0    11.0   120.0
Max              160.0      6.0  5.0   320.0  14.00   23.0    15.0   330.0

              vitamins  shelf  weight  cups     rating
Min                0.0    1.0     0.5  0.25  18.042851
Q1 (25%)          25.0    1.0     1.0  0.67  32.932466
Median (50%)      25.0    2.0     1.0  0.75  40.253086
Q3 (75%)          25.0    3.0     1.0  1.00  50.780847
Max              100.0    3.0     1.5  1.50  93.704912
```

2) For each of the numeric attributes (proteins up to vitamins), identify and replace all

missing data (indicated with -1) with the arithmetic mean of the attribute.

In [16]:
```
import pandas as pd
df = pd.read_csv(r"C:\Prashma\AI-ML Data set\Cereals1.csv")
numeric_cols = ['protein', 'fat', 'sodium', 'fiber', 'carbo', 'sugars', 'potas
df[numeric_cols] = df[numeric_cols].astype(float)
# Replace -1 with mean of the column
```

```
for col in numeric_cols:
    mean_val = df.loc[df[col] != -1, col].mean()
    df.loc[df[col] == -1, col] = mean_val

print(df[numeric_cols].head(25))
```

```
    protein  fat  sodium  fiber  carbo  sugars      potass  vitamins
0       3.0  5.0    15.0    2.0    8.0     8.0  135.000000       0.0
1       4.0  1.0   260.0    9.0    7.0     5.0  320.000000      25.0
2       4.0  0.0   140.0   14.0    8.0     0.0  330.000000      25.0
3       2.0  2.0   200.0    1.0   14.0     8.0   96.216216      25.0
4       2.0  2.0   180.0    1.5   10.5    10.0   70.000000      25.0
5       2.0  0.0   125.0    1.0   11.0    14.0   30.000000      25.0
6       3.0  2.0   210.0    2.0   18.0     8.0  100.000000      25.0
7       2.0  1.0   200.0    4.0   15.0     6.0  125.000000      25.0
8       3.0  0.0   210.0    5.0   13.0     5.0  190.000000      25.0
9       1.0  2.0   220.0    0.0   12.0    12.0   35.000000      25.0
10      6.0  2.0   290.0    2.0   17.0     1.0  105.000000      25.0
11      1.0  3.0   210.0    0.0   13.0     9.0   45.000000      25.0
12      3.0  2.0   140.0    2.0   13.0     7.0  105.000000      25.0
13      1.0  1.0   180.0    0.0   12.0    13.0   55.000000      25.0
14      2.0  0.0   280.0    0.0   22.0     3.0   25.000000      25.0
15      2.0  0.0   290.0    1.0   21.0     2.0   35.000000      25.0
16      1.0  0.0    90.0    1.0   13.0    12.0   20.000000      25.0
17      1.0  1.0   180.0    0.0   12.0    13.0   65.000000      25.0
18      3.0  3.0   140.0    4.0   10.0     7.0  160.000000      25.0
19      3.0  0.0    80.0    1.0   21.0     0.0   96.216216       0.0
20      2.0  0.0   220.0    1.0   21.0     3.0   30.000000      25.0
21      2.0  1.0   140.0    2.0   11.0    10.0  120.000000      25.0
22      2.0  0.0   190.0    1.0   18.0     5.0   80.000000      25.0
23      2.0  1.0   125.0    1.0   11.0    13.0   30.000000      25.0
24      1.0  0.0   200.0    1.0   14.0    11.0   25.000000      25.0
```

3) Create a table with the 5-number summary of all the numeric attributes after treating

missing values. Do you think the strategy used in dealing with missing values was

effective?

In [18]:
```
import pandas as pd
df = pd.read_csv(r"C:\Prashma\AI-ML Data set\Cereals1.csv")

numeric_cols = ['protein', 'fat', 'sodium', 'fiber', 'carbo', 'sugars', 'potas
df[numeric_cols] = df[numeric_cols].astype(float)
for col in numeric_cols:
    mean_val = df.loc[df[col] != -1, col].mean()
    df.loc[df[col] == -1, col] = mean_val

numeric_df = df.select_dtypes(include='number')
```

```
summary = numeric_df.describe().loc[['min', '25%', '50%', '75%', 'max']]
summary.index = ['Min', 'Q1 (25%)', 'Median (50%)', 'Q3 (75%)', 'Max']

print("5-Number Summary after missing value treatment:")
print(summary)
```

```
5-Number Summary after missing value treatment:
              calories  protein  fat  sodium  fiber      carbo  sugars  \
Min               50.0      1.0  0.0     0.0   0.00   7.000000     0.0
Q1 (25%)         100.0      2.0  0.0   132.5   0.75  12.000000     3.0
Median (50%)     110.0      2.5  1.0   180.0   1.75  14.966667     7.0
Q3 (75%)         110.0      3.0  2.0   212.5   3.00  17.000000    11.0
Max              160.0      6.0  5.0   320.0  14.00  23.000000    15.0

              potass  vitamins  shelf  weight  cups     rating
Min            15.00       0.0    1.0     0.5  0.25  18.042851
Q1 (25%)       43.75      25.0    1.0     1.0  0.67  32.932466
Median (50%)   90.00      25.0    2.0     1.0  0.75  40.253086
Q3 (75%)      120.00      25.0    3.0     1.0  1.00  50.780847
Max           330.00     100.0    3.0     1.5  1.50  93.704912
```

4) For each of the numeric attributes (proteins up to vitamins), identify and replace all

noisy data with the median of attribute.

In [19]:

```
import pandas as pd

df = pd.read_csv(r"C:\Prashma\AI-ML Data set\Cereals1.csv")

numeric_cols = ['protein', 'fat', 'sodium', 'fiber', 'carbo', 'sugars', 'potas
df[numeric_cols] = df[numeric_cols].astype(float)
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    median_val = df[col].median()

    df.loc[(df[col] < lower_bound) | (df[col] > upper_bound), col] = median_va

print(df[numeric_cols].head())
```

```
   protein  fat  sodium  fiber  carbo  sugars  potass  vitamins
0      3.0  5.0    15.0   2.00    8.0     8.0   135.0      25.0
1      4.0  1.0   260.0   1.75    7.0     5.0    90.0      25.0
2      4.0  0.0   140.0   1.75    8.0     0.0    90.0      25.0
3      2.0  2.0   200.0   1.00   14.0     8.0    -1.0      25.0
4      2.0  2.0   180.0   1.50   10.5    10.0    70.0      25.0
```

5) Create a table with the 5-number summary of all the numeric attributes after treating

noisy values. Do you think the strategy used in dealing with noisy values was

effective?

In [21]:
```
import pandas as pd

df = pd.read_csv(r"C:\Prashma\AI-ML Data set\Cereals1.csv")

numeric_cols = ['protein', 'fat', 'sodium', 'fiber', 'carbo', 'sugars', 'potas
df[numeric_cols] = df[numeric_cols].astype(float)
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    median_val = df[col].median()

    df.loc[(df[col] < lower_bound) | (df[col] > upper_bound), col] = median_va

numeric_df = df.select_dtypes(include='number')
summary = numeric_df.describe().loc[['min', '25%', '50%', '75%', 'max']]
summary.index = ['Min', 'Q1 (25%)', 'Median (50%)', 'Q3 (75%)', 'Max']

print("5-Number Summary after treating noisy values:")
print(summary)
```

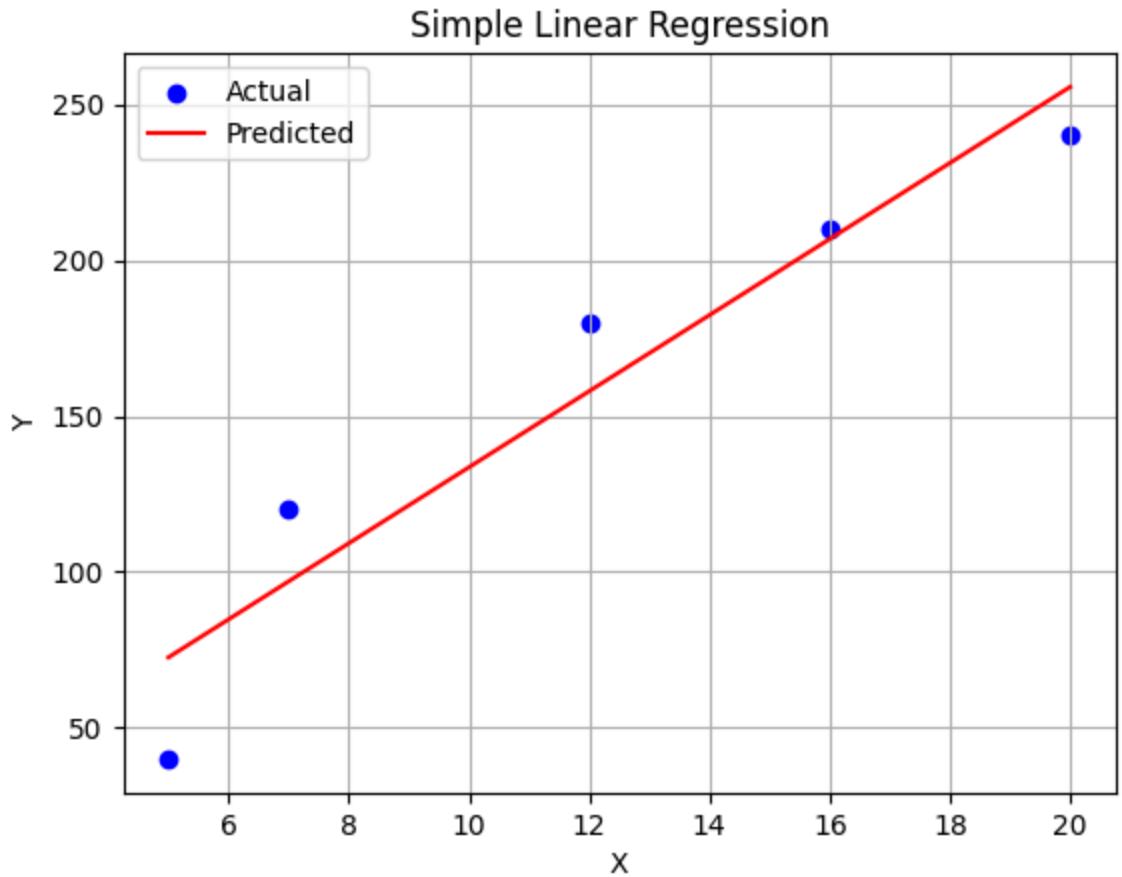
```
5-Number Summary after treating noisy values:
               calories  protein  fat  sodium  fiber  carbo  sugars  potass  \
Min                50.0     1.00  0.0    15.0  0.000   7.00    -1.0    -1.0
Q1 (25%)          100.0     2.00  0.0   147.5  0.750  12.00     3.0    40.0
Median (50%)      110.0     2.25  1.0   180.0  1.625  14.75     7.0    90.0
Q3 (75%)          110.0     3.00  2.0   212.5  3.000  17.00    11.0   110.0
Max               160.0     4.00  5.0   320.0  6.000  23.00    15.0   240.0

               vitamins  shelf  weight  cups     rating
Min                25.0    1.0     0.5  0.25  18.042851
Q1 (25%)           25.0    1.0     1.0  0.67  32.932466
Median (50%)       25.0    2.0     1.0  0.75  40.253086
Q3 (75%)           25.0    3.0     1.0  1.00  50.780847
Max                25.0    3.0     1.5  1.50  93.704912
```

Problems based program

Simple linear Regression(Using Library function)

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Sample data
x = np.array([5, 7, 12, 16, 20]).reshape(-1, 1)  # Independent variable
y = np.array([40, 120, 180, 210, 240])          # Dependent variable

# Create and train the model
model = LinearRegression()
model.fit(x, y)

# Predict values
y_pred = model.predict(x)

# Print model parameters
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)

# Calculate error metrics
mse = mean_squared_error(y, y_pred)
mae = mean_absolute_error(y, y_pred)
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Simple Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Slope (Coefficient): 12.20779220779221
Intercept: 11.506493506493484
Mean Squared Error (MSE): 465.87012987012986
Mean Absolute Error (MAE): 19.283116883116882
Root Mean Squared Error (RMSE): 21.58402487651758
```

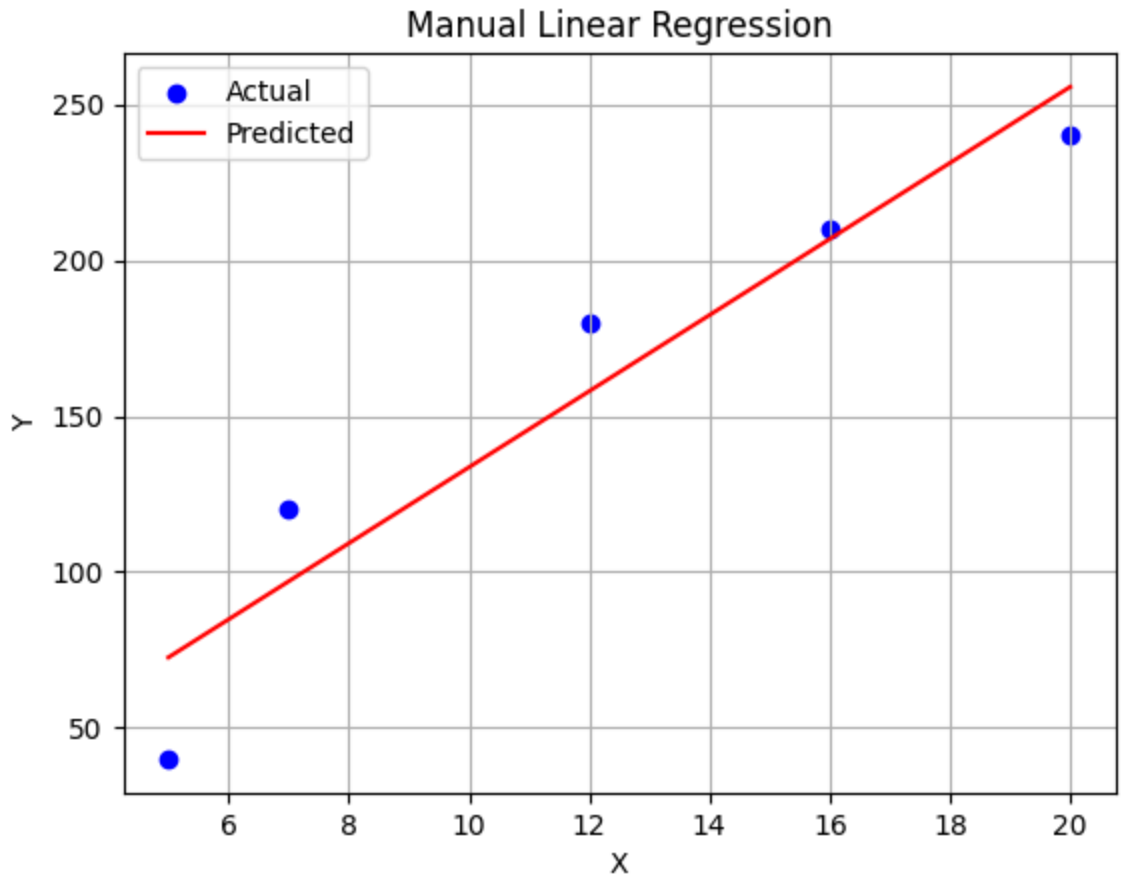


simple linear function without Library function

In [15]:
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([5, 7, 12, 16, 20])
y = np.array([40, 120, 180, 210, 240])

# Step 1: Calculate slope (m) and intercept (b) manually
n = len(x)
x_mean = np.mean(x)
y_mean = np.mean(y)

# Calculate slope (m)
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean) ** 2)
slope = numerator / denominator

# Calculate intercept (b)
intercept = y_mean - slope * x_mean
```

```python
print("Slope (Coefficient):", slope)
print("Intercept:", intercept)

# Step 2: Predict values
y_pred = slope * x + intercept

# Step 3: Calculate error metrics manually
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 4: Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Manual Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Slope (Coefficient): 12.207792207792208
Intercept: 11.506493506493513
Mean Squared Error (MSE): 465.87012987013014
Mean Absolute Error (MAE): 19.283116883116886
Root Mean Squared Error (RMSE): 21.58402487651759
```



Simple Linear Regression(using Matrix method)

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([5, 7, 12, 16, 20])
y = np.array([40, 120, 180, 210, 240])

# Step 1: Prepare the design matrix X and target vector Y
X = np.vstack([np.ones(len(x)), x]).T  # Add intercept term
Y = y.reshape(-1, 1)

# Step 2: Compute coefficients using the Normal Equation: θ = (XᵀX)⁻¹XᵀY
theta = np.linalg.inv(X.T @ X) @ X.T @ Y
intercept, slope = theta.flatten()

print("Slope (Coefficient):", slope)
print("Intercept:", intercept)

# Step 3: Predict values
y_pred = X @ theta
```

```
y_pred = y_pred.flatten()

# Step 4: Calculate error metrics
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 5: Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Regression (Matrix Method)')
plt.legend()
plt.grid(True)
plt.show()
```

Multiple linear Regression(Using Library function)

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Sample data: Two independent variables (X1 and X2)
# Shape: (number of samples, number of features)
X = np.array([
    [1, 10],
    [3, 14],
    [4, 15],
    [6, 18],
    [7, 20]
])

# Dependent variable
y = np.array([9, 10, 13, 14, 16])

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# Predict values
y_pred = model.predict(X)

# Print model parameters
```

```
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Optional: Plot actual vs predicted
plt.scatter(range(len(y)), y, color='blue', label='Actual')
plt.plot(range(len(y_pred)), y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

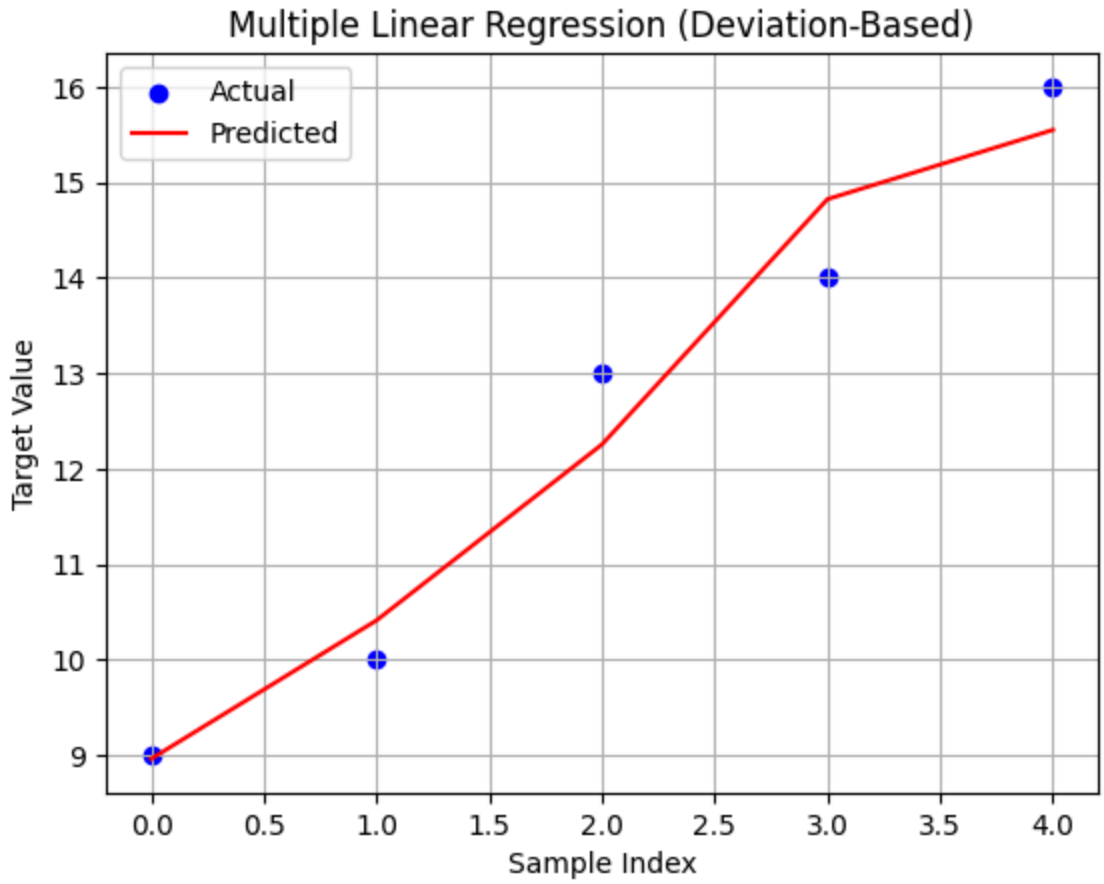
multiple linear regression (without library function)

In [20]:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
X1 = np.array([1, 3, 4, 6, 7])
X2 = np.array([10, 14, 15, 18, 20])
Y = np.array([9, 10, 13, 14, 16])
n = len(Y)

# Step 1: Compute required summations
sum_X1 = np.sum(X1)
sum_X2 = np.sum(X2)
sum_Y = np.sum(Y)

sum_X1_squared = np.sum(X1 ** 2)
sum_X2_squared = np.sum(X2 ** 2)
sum_X1X2 = np.sum(X1 * X2)
sum_X1Y = np.sum(X1 * Y)
sum_X2Y = np.sum(X2 * Y)

# Step 2: Compute deviation-based terms
S_x1_squared = sum_X1_squared - (sum_X1 ** 2) / n
S_x2_squared = sum_X2_squared - (sum_X2 ** 2) / n
S_x1x2 = sum_X1X2 - (sum_X1 * sum_X2) / n
S_x1y = sum_X1Y - (sum_X1 * sum_Y) / n
S_x2y = sum_X2Y - (sum_X2 * sum_Y) / n

# Step 3: Compute coefficients using your formulas
denominator = (S_x1_squared * S_x2_squared) - (S_x1x2 ** 2)

beta_1 = ((S_x2_squared * S_x1y) - (S_x1x2 * S_x2y)) / denominator
beta_2 = ((S_x1_squared * S_x2y) - (S_x1x2 * S_x1y)) / denominator

# Step 4: Compute intercept
```

```python
X1_mean = np.mean(X1)
X2_mean = np.mean(X2)
Y_mean = np.mean(Y)

beta_0 = Y_mean - beta_1 * X1_mean - beta_2 * X2_mean

print("Intercept (β₀):", beta_0)
print("Coefficient β₁ (for X₁):", beta_1)
print("Coefficient β₂ (for X₂):", beta_2)

# Step 5: Predict values
Y_pred = beta_0 + beta_1 * X1 + beta_2 * X2

# Step 6: Error metrics
mse = np.mean((Y - Y_pred) ** 2)
mae = np.mean(np.abs(Y - Y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 7: Plot actual vs predicted
plt.scatter(range(n), Y, color='blue', label='Actual')
plt.plot(range(n), Y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression (Deviation-Based)')
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept (β₀): 17.176470588236597
Coefficient β₁ (for X₁): 2.960784313725723
Coefficient β₂ (for X₂): -1.1176470588236775
Mean Squared Error (MSE): 0.32156862745097836
Mean Absolute Error (MAE): 0.4941176470588065
Root Mean Squared Error (RMSE): 0.5670702138633085
```



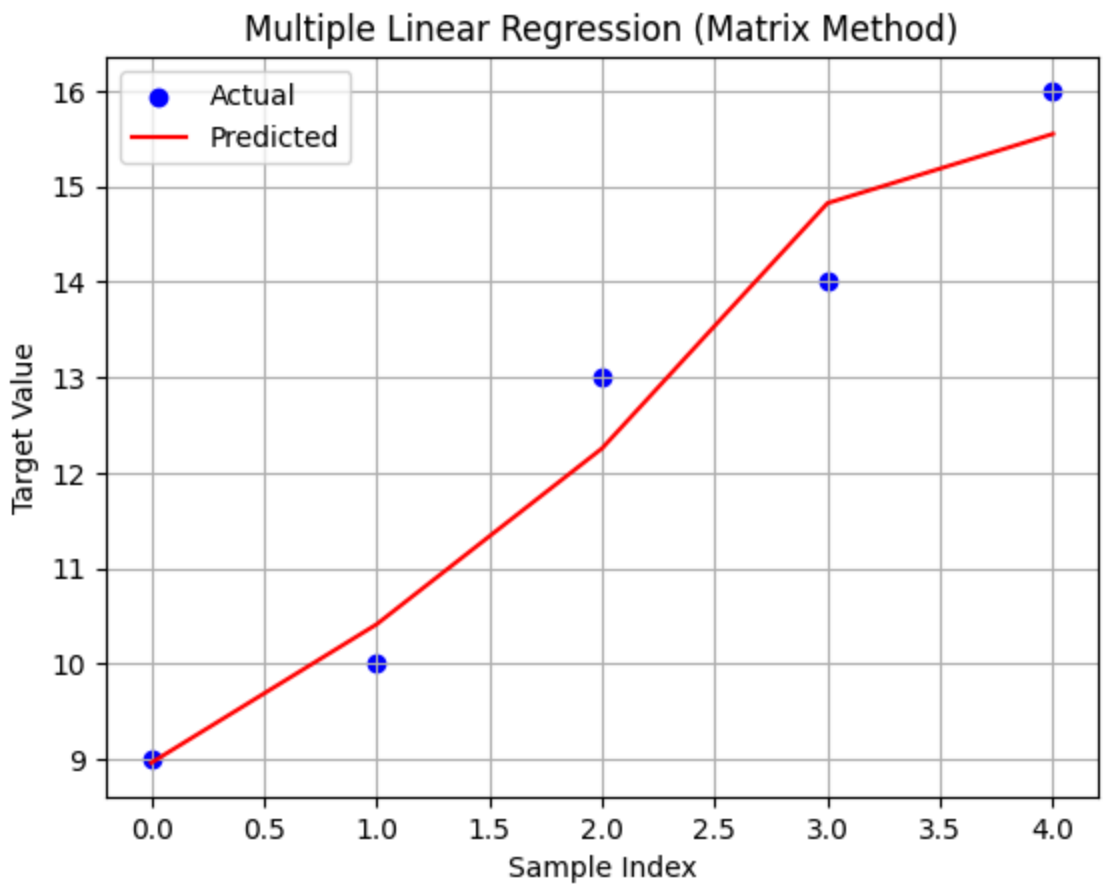
multiple linear regression (matrix method)

In [13]:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data: Two independent variables (X1 and X2)
X = np.array([
    [1, 10],
    [3, 14],
    [4, 15],
    [6, 18],
    [7, 20]
])

# Dependent variable
y = np.array([9, 10, 13, 14, 16])

# Step 1: Add intercept term (column of ones)
X_design = np.hstack((np.ones((X.shape[0], 1)), X))  # Shape: (n_samples, n_fe

# Step 2: Compute coefficients using Normal Equation: θ = (XᵀX)⁻¹Xᵀy
theta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ y
```

```python
# Extract intercept and coefficients
intercept = theta[0]
coefficients = theta[1:]

print("Intercept:", intercept)
print("Coefficients:", coefficients)

# Step 3: Predict values
y_pred = X_design @ theta

# Step 4: Calculate error metrics manually
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 5: Plot actual vs predicted
plt.scatter(range(len(y)), y, color='blue', label='Actual')
plt.plot(range(len(y_pred)), y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression (Matrix Method)')
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept: 17.176470588235418
Coefficients: [ 2.96078431 -1.11764706]
Mean Squared Error (MSE): 0.32156862745098036
Mean Absolute Error (MAE): 0.4941176470587386
Root Mean Squared Error (RMSE): 0.5670702138633102
```

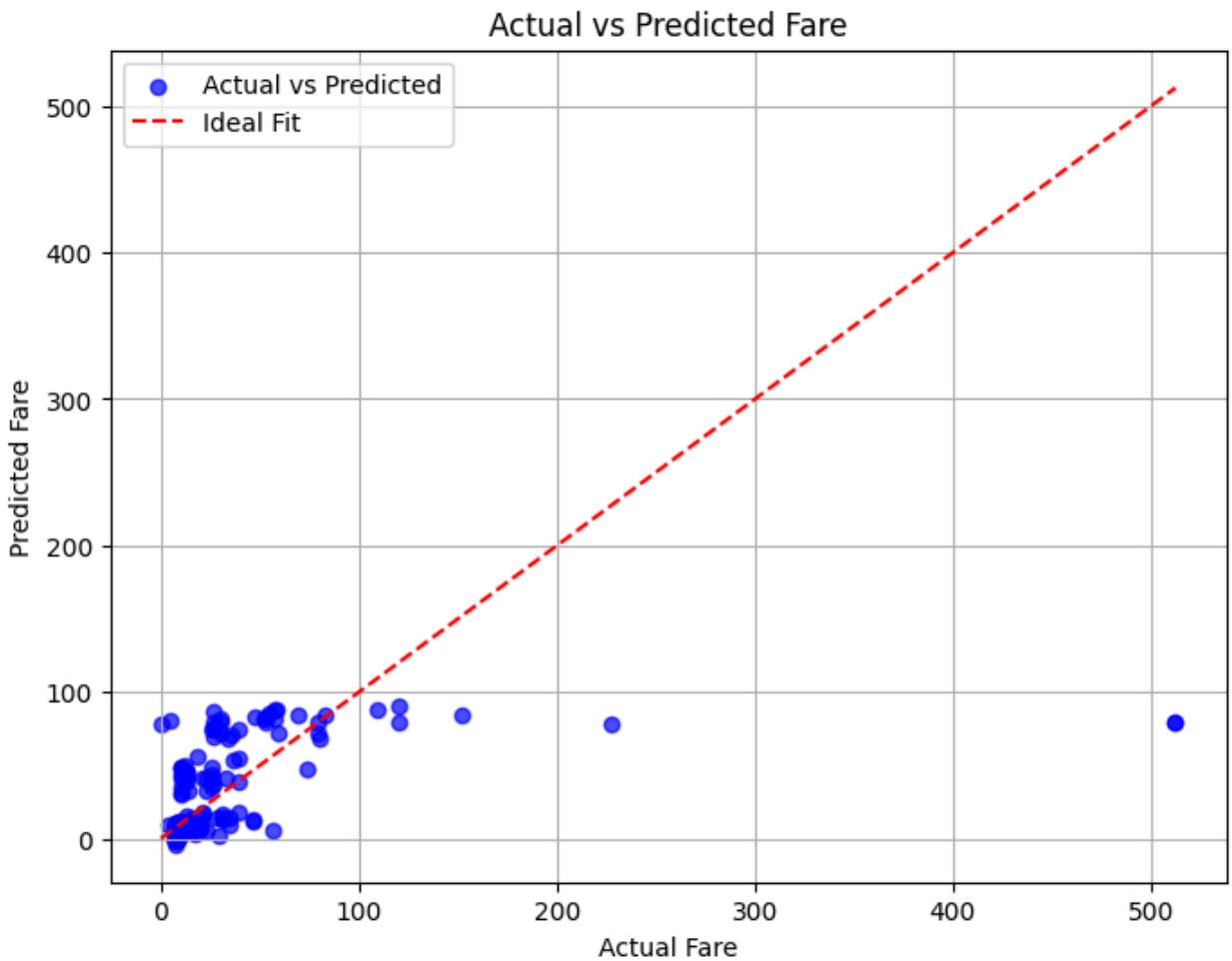


USING titanic DATASET

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split  # Corrected import

# Load and clean data
tit = pd.read_csv(r"C:\Prashma\AI-ML Data set\AI&ML_Dataset\titanic.csv")
tit = tit.dropna(subset=['Age', 'Fare', 'Pclass'])

# Feature and target selection
x = tit[['Age', 'Pclass']]
y = tit['Fare']

# Train-test split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc

# Model training
model = LinearRegression()
model.fit(x_train, y_train)
```

```python
# Prediction
y_pred = model.predict(x_test)

# Evaluation
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Output model parameters
print("Intercept (a):", model.intercept_)
print("Coefficient for Age (b1):", model.coef_[0])
print("Coefficient for PClass (b2):", model.coef_[1])
print("Mean Squared Error:", mse)
print("R² Score:", r2)

# Visualization
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.7, label="Actual vs Predicte
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='re
plt.title('Actual vs Predicted Fare')
plt.xlabel('Actual Fare')
plt.ylabel('Predicted Fare')
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept (a): 133.3072666812878
Coefficient for Age (b1): -0.4369588850881898
Coefficient for PClass (b2): -38.297289346757964
Mean Squared Error: 3364.922912856981
R² Score: 0.1917633318253048
```



Polynomial (Quadratic regression model for the following data)-using matrix method using library function

In [2]:
```
import numpy as np

# Data
x = np.array([3, 4, 5, 6, 7])
y = np.array([2.5, 3.2, 3.8, 6.5, 11.5])

# Fit a 2nd-degree polynomial: y = a2*x^2 + a1*x + a0
coeffs = np.polyfit(x, y, deg=2)  # Returns [a2, a1, a0]
a2, a1, a0 = coeffs

# Print the full formula without using f-string
print("y =", a2, "x² +", a1, "x +", a0)
```

```
y = 0.7642857142857119 x² + -5.512857142857122 x + 12.428571428571374
```

Using Dataset Iris

In [3]:
```
import numpy as np
import pandas as pd

# Load dataset
dataset_path = r"C:\Prashma\AI-ML Data set\AI&ML_Dataset\Iris.csv"
data = pd.read_csv(dataset_path)

# Extract x and y from dataset
x = data['Id'].values
y = data['PetalLengthCm'].values

# Fit a 2nd-degree polynomial: y = a2*x^2 + a1*x + a0
coeffs = np.polyfit(x, y, deg=2)  # Returns [a2, a1, a0]
a2, a1, a0 = coeffs

# Print the full formula without using f-string
print("y =", a2, "x² +", a1, "x +", a0)
```

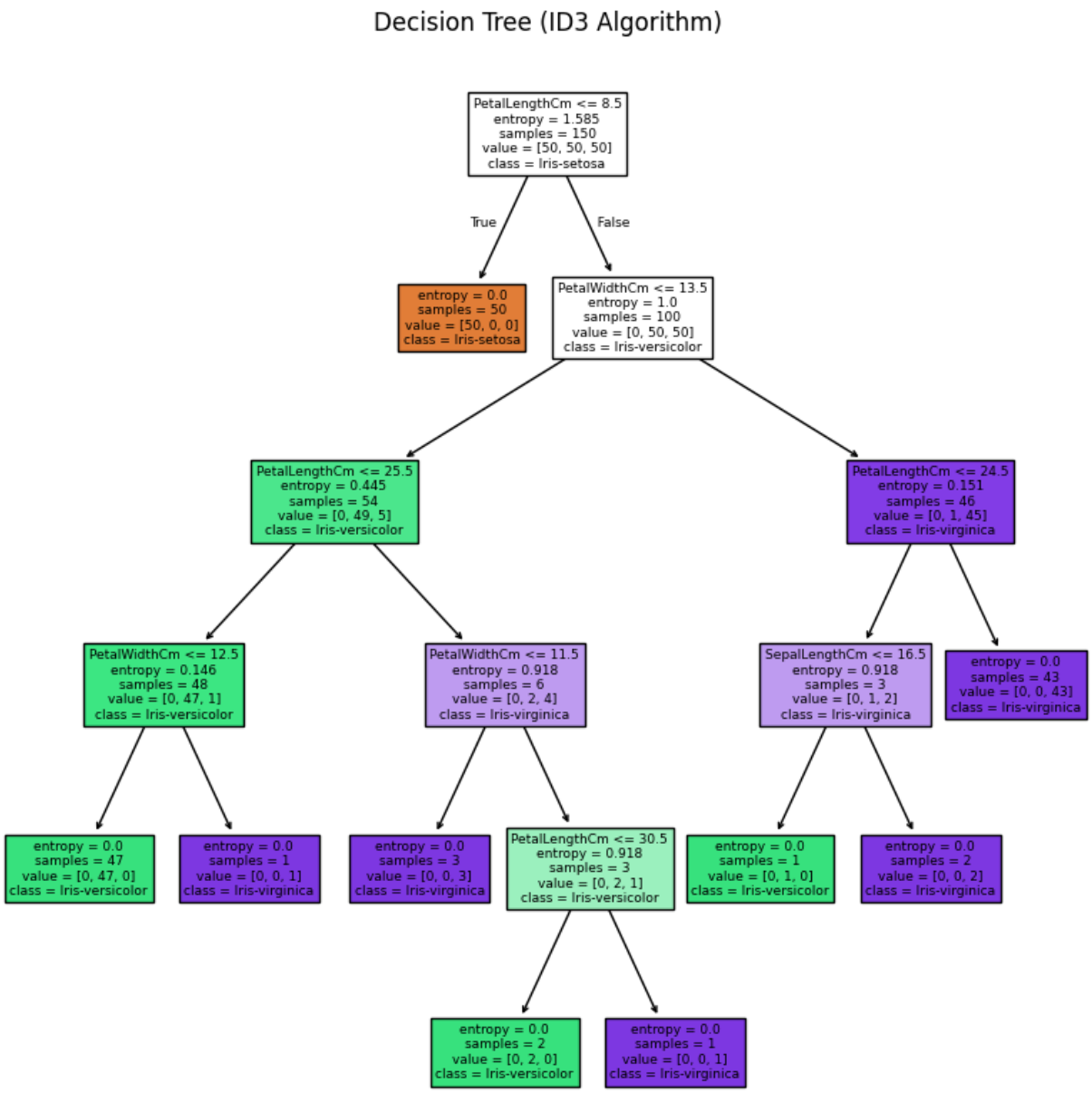
```
y = -0.00017214057308821763 x² + 0.06184370877997612 x + 0.3934601850172305
```

In [ ]:

IRIS DATASET

Decision Tree ID3

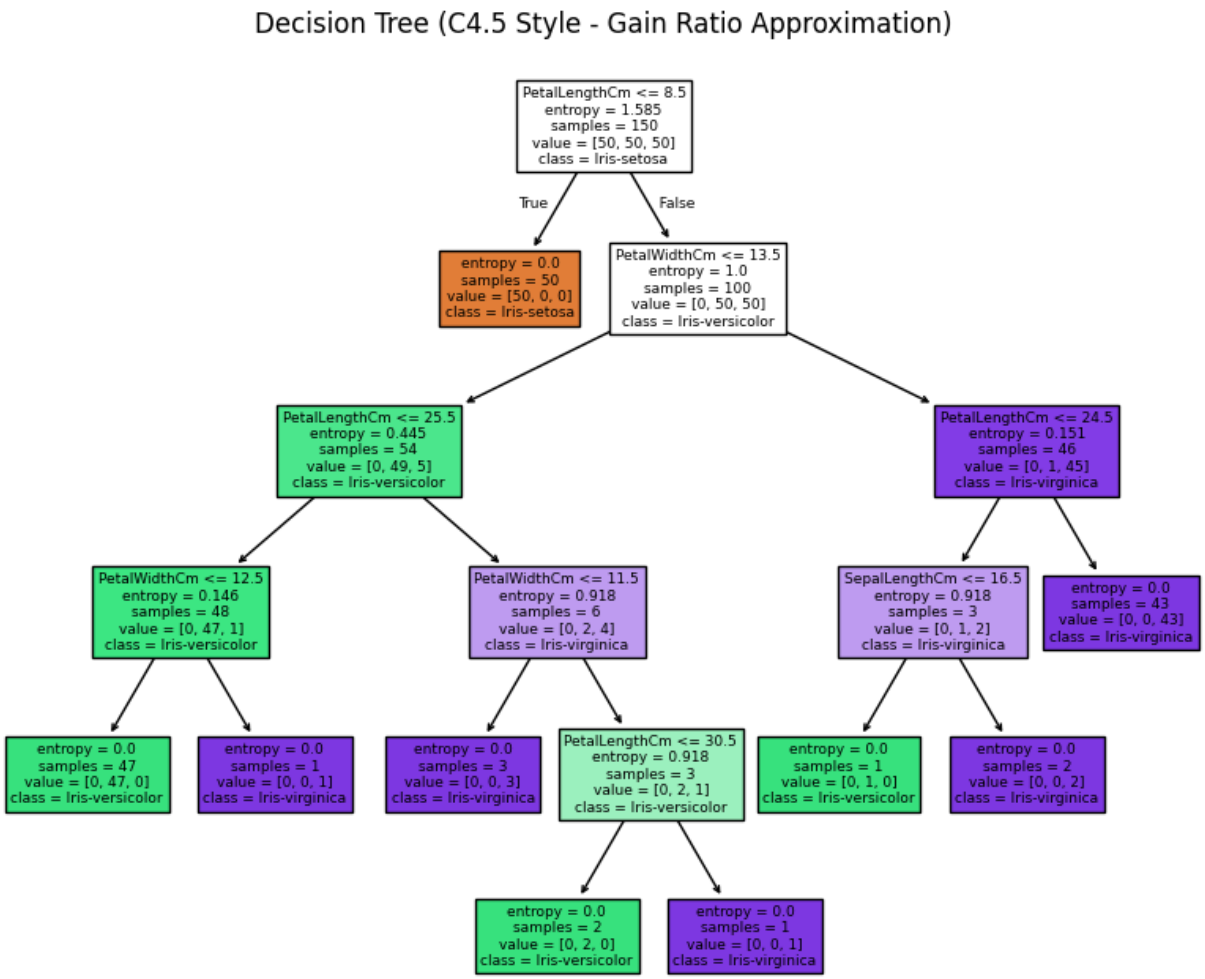
In [4]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
# Load the dataset
df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\Iris.csv')
df.drop(columns=["Id"], inplace=True)
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])
X = df.drop(columns=["Species"])
y = df["Species"]
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
plt.figure(figsize=(10, 10))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (ID3 Algorithm)")
plt.show()
```



Decision Tree C4.5

In [5]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load and preprocess the dataset
df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\Iris.csv')
df.drop(columns=["Id"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
```

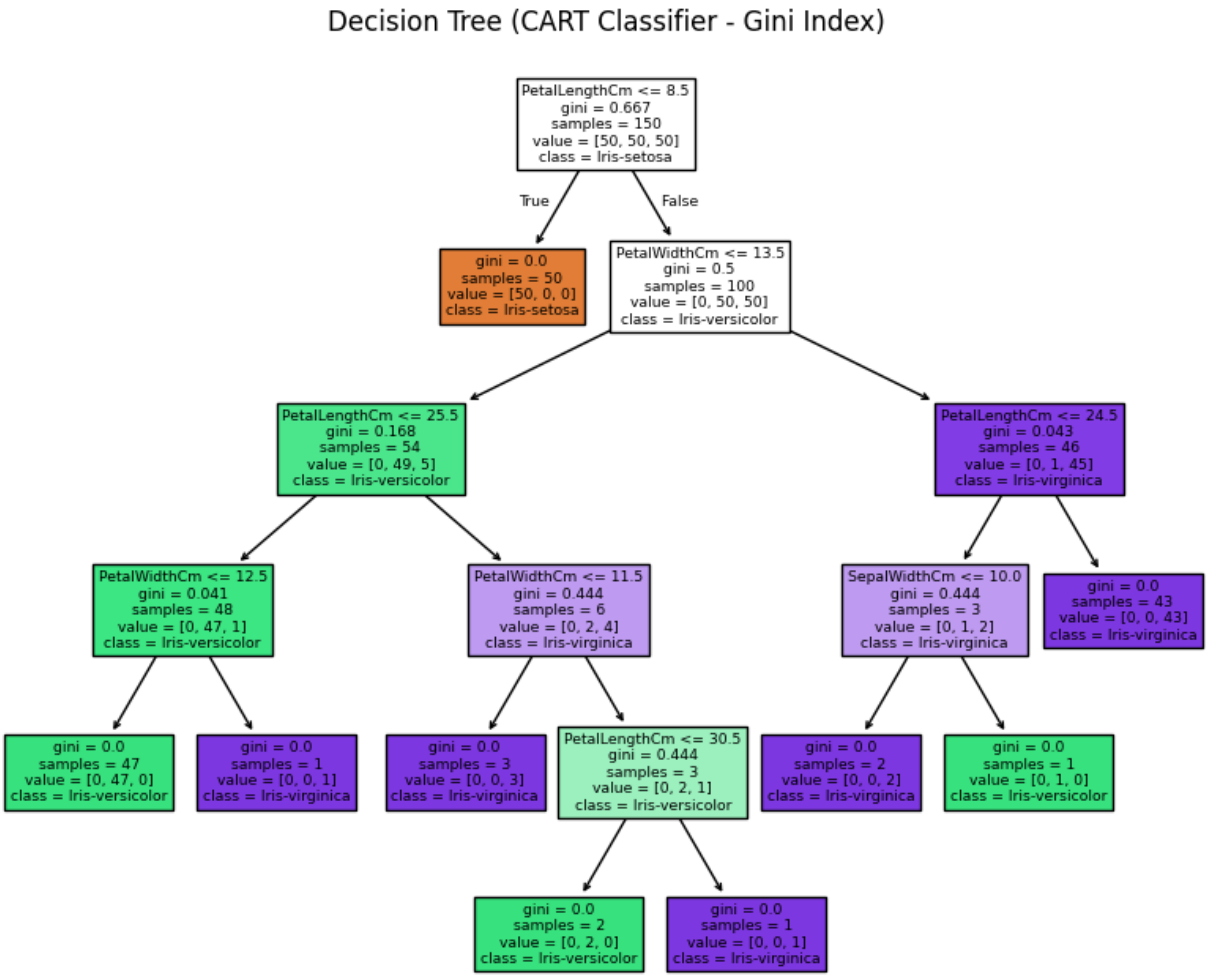
```
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Species"])
y = df["Species"]

# Train decision tree using entropy (C4.5-style
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (C4.5 Style - Gain Ratio Approximation)")
plt.show()
```



Decision Tree Cart classifier

In [6]:
```
import pandas as pd
```

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\Iris.csv')
df.drop(columns=["Id"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Species"])
y = df["Species"]

# Train decision tree using CART (Gini index)
model = DecisionTreeClassifier(criterion="gini")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (CART Classifier - Gini Index)")
plt.show()
```

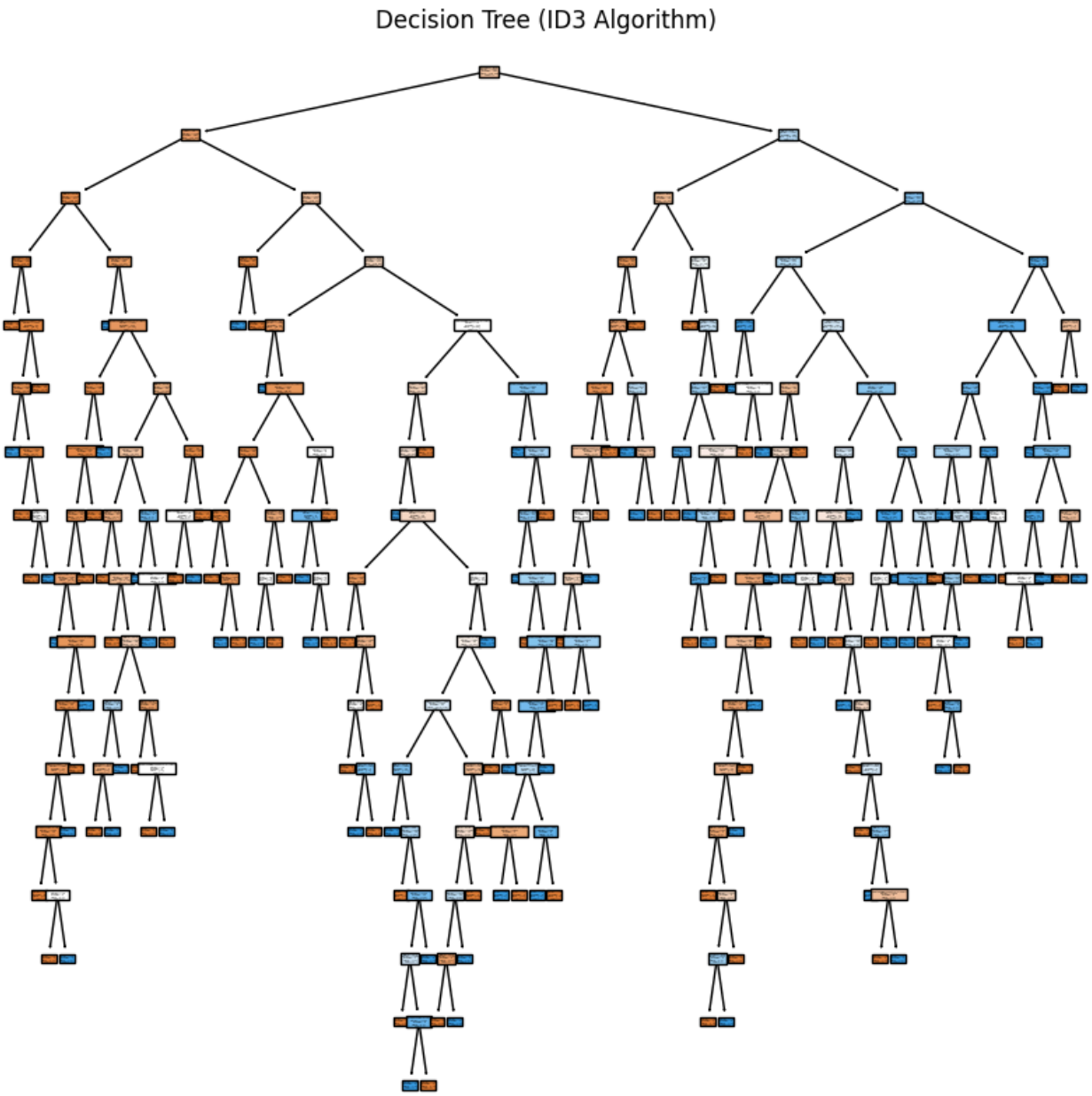


Diabities data set

ID3

In [9]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
# Load the dataset
df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\diabetes.csv')
df.drop(columns=["Pregnancies"], inplace=True)
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])
X = df.drop(columns=["Outcome"])
y = df["Outcome"]
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
```

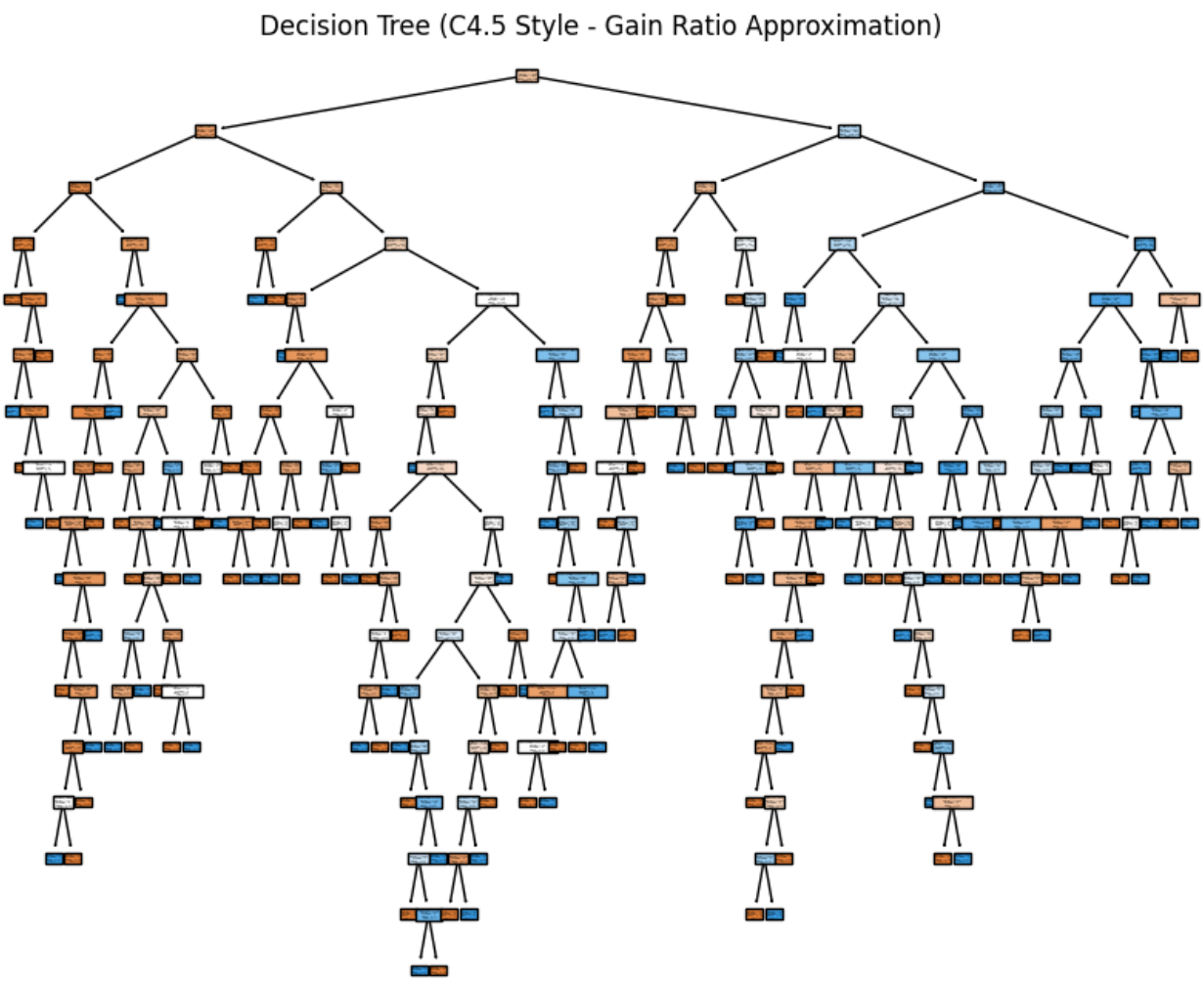
```
plt.figure(figsize=(10, 10))
plot_tree(model, feature_names=X.columns, class_names=["0", "1"], filled=True)
plt.title("Decision Tree (ID3 Algorithm)")
plt.show()
```



c4.5

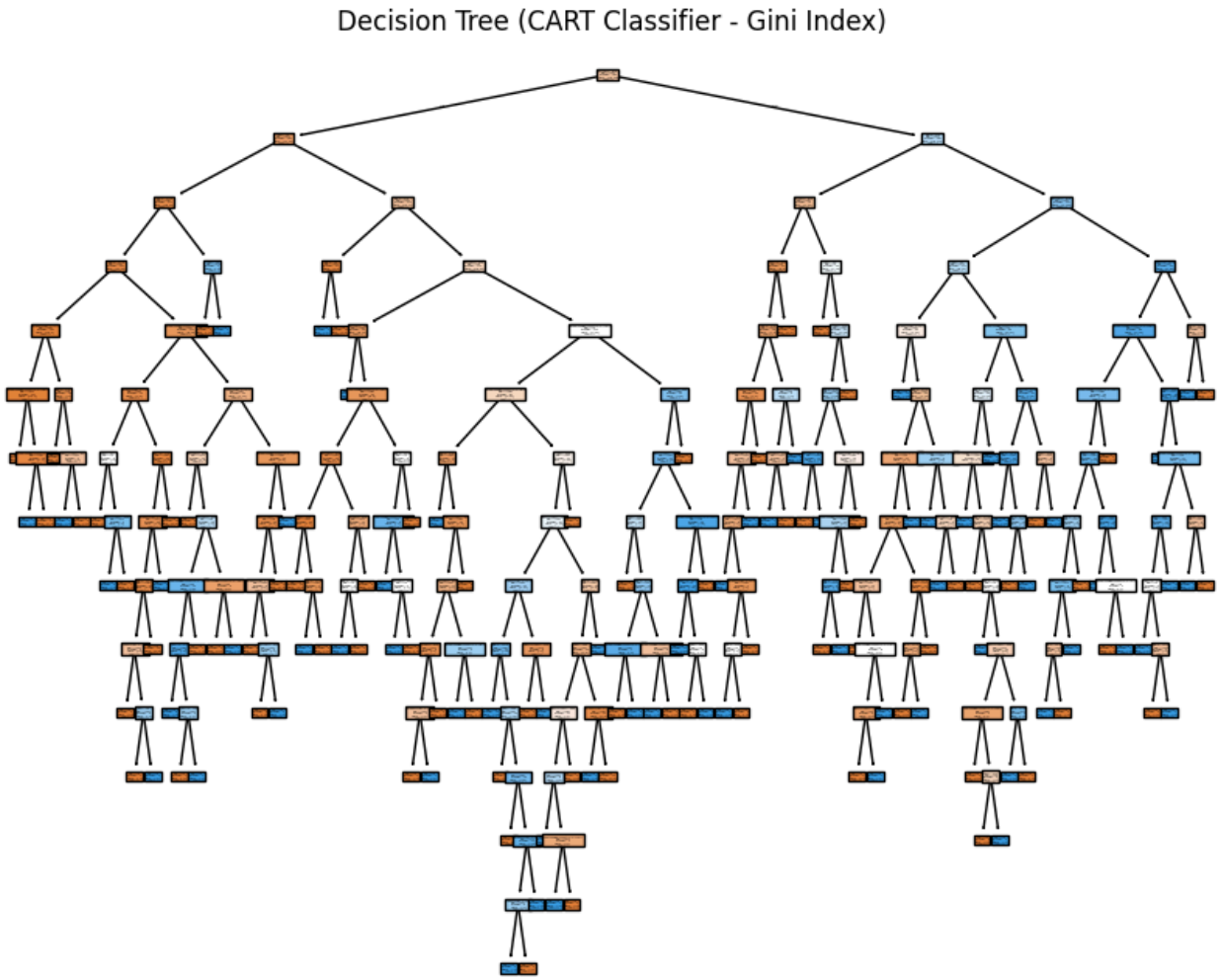
In [10]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load and preprocess the dataset
```

```python
df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\diabetes.csv')
df.drop(columns=["Pregnancies"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Train decision tree using entropy (C4.5-style
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["0", "1"], filled=True)
plt.title("Decision Tree (C4.5 Style - Gain Ratio Approximation)")
plt.show()
```



Cart Classifier

In [12]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv(r'C:\Prashma\AI&ML_Dataset\diabetes.csv')
df.drop(columns=["Pregnancies"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Train decision tree using CART (Gini index)
model = DecisionTreeClassifier(criterion="gini")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["0", "1"], filled=True)
plt.title("Decision Tree (CART Classifier - Gini Index)")
plt.show()
```



In [ ]: